

Assignment 4 - Ibrahim Zbib

Ibrahim Zbib - K nearest neighbors

```
In [1]: import os, glob, warnings
import numpy as np
import pandas as pd
from pathlib import Path

from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_auc_score, roc_curve, auc, confusion_matrix
from sklearn.base import BaseEstimator, TransformerMixin

import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display # <-- ensure plots appear in execution

DATA_DIR = Path("data/SRP192714")
RESULTS_DIR = Path("results")
PLOTS_DIR = Path("plots")
RESULTS_DIR.mkdir(exist_ok=True)
PLOTS_DIR.mkdir(exist_ok=True)

from pathlib import Path
Path("reports").mkdir(parents=True, exist_ok=True)

# Track only plots created by THIS notebook:
PLOT_FILES = []

import itertools
_counter = itertools.count(1)
def savefig_pdf(prefix="plot"):
    import matplotlib.pyplot as plt
    n = next(_counter)
    plt.gcf().savefig(f"reports/{prefix}_{n:02d}.pdf", bbox_inches="tight")

# assignment 2 artifacts
EXPR_PATH = DATA_DIR / "SRP192714.tsv"
META_PATH = DATA_DIR / "metadata_SRP192714_merged.tsv"

def read_tsv_safe(path):
    try:
        return pd.read_csv(path, sep="\t")
    except UnicodeDecodeError:
        warnings.warn(f"unicode error on {path}, using latin-1")
        return pd.read_csv(path, sep="\t", encoding="latin-1")
```

```

# load a2 outputs
expr_df = read_tsv_safe(EXPR_PATH)
meta_df = read_tsv_safe(META_PATH)

# make samples x genes matrix
assert "Gene" in expr_df.columns, "expected 'Gene' column in expression data"
expr_df = expr_df.set_index("Gene").apply(pd.to_numeric, errors="coerce")
X_full = expr_df.T.copy()
X_full.index.name = "sample_id"

# build assignment-1 binary labels from exposure
def to_exposure2(x):
    s = str(x).lower()
    if s in {"naive", "mock", "control", "uninfected", "healthy"}: return 0
    if ("denv" in s) or ("dengue" in s): return "DENV"
    return np.nan

for col in ["refinebio_accession_code", "Exposure2"]:
    assert col in meta_df.columns, f"missing '{col}' in metadata"
meta_df["Exposure2"] = meta_df["Exposure"].map(to_exposure2)
meta_df = meta_df.dropna(subset=["Exposure2"]).copy()
meta_df = meta_df.set_index("refinebio_accession_code").sort_index()

# align samples
common_ids = sorted(set(X_full.index) & set(meta_df.index))
assert len(common_ids) > 0, "no overlapping samples"
X_full = X_full.loc[common_ids]
y_bin = meta_df.loc[common_ids, "Exposure2"].astype("category")

# feature selector: top-k by variance
class TopKVariance(BaseEstimator, TransformerMixin):
    def __init__(self, k=5000):
        self.k = k
        self.cols_ = None
        self._use_names = True
    def fit(self, X, y=None):
        n_feats = X.shape[1]
        k = min(self.k, n_feats)
        if isinstance(X, pd.DataFrame):
            variances = X.var(axis=0)
            self.cols_ = variances.nlargest(k).index.tolist()
            self._use_names = True
        else:
            variances = np.var(X, axis=0)
            self.cols_ = np.argsort(variances)[::-1][:k].tolist()
            self._use_names = False
        return self
    def transform(self, X):
        if isinstance(X, pd.DataFrame):
            return X[self.cols_] if self._use_names else X.iloc[:, self.cols_]
        return X[:, self.cols_]
    def get_feature_names_out(self):

```

```

        return np.asarray(self.cols_)

def build_knn_pipeline(k_genes):
    pipe = Pipeline(steps=[
        ("topk", TopKVariance(k=k_genes)),
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier())
    ])
    param_grid = {
        "knn__n_neighbors": [1,3,5,7,9,11,15,21,31,41,51],
        "knn__weights": ["uniform","distance"],
        "knn__p": [1,2]
    }
    return pipe, param_grid

# evaluate binary auc with cv and grid search
def evaluate_binary_auc(X, y, k_genes, cv_splits=5, seed=42, n_jobs=-1):
    pipe, grid = build_knn_pipeline(k_genes)
    cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=seed)
    clf = GridSearchCV(pipe, grid, scoring="roc_auc", cv=cv, n_jobs=n_jobs)
    clf.fit(X, y)
    best = clf.best_estimator_
    prob_cv = cross_val_predict(best, X, y, cv=cv, method="predict_proba")
    denv_index = list(best.named_steps["knn"].classes_).index("DENV")
    y_prob_denv = prob_cv[:, denv_index]
    y_pred = (y_prob_denv >= 0.5).astype(int)
    y_true = (y == "DENV").astype(int)
    auc_mean = float(clf.best_score_)
    auc_cv_reassembled = float(roc_auc_score(y_true, y_prob_denv))
    cm = confusion_matrix(y_true, y_pred)
    rep = classification_report(y_true, y_pred, target_names=["Naive", "DENV"])
    return {
        "best_params": clf.best_params_,
        "best_auc": auc_mean,
        "holdout_like_auc": auc_cv_reassembled,
        "confusion_matrix": cm,
        "report": rep,
        "best_estimator": best,
        "prob_cv": y_prob_denv,
        "pred_labels": np.where(y_pred==1, "DENV", "Naive")
    }

# baseline model at 5k genes
bin_5000 = evaluate_binary_auc(X_full, y_bin, k_genes=5000)
print("a1/binary - 5k genes -> mean-cv auc:", bin_5000["best_auc"])
print("best params:", bin_5000["best_params"])
print(bin_5000["report"])

# save per-sample predictions
pd.DataFrame({
    "sample_id": X_full.index,
    "true_label": y_bin.values,
    "pred_label": y_pred
})

```

```

    "knn_pred_label": bin_5000["pred_labels"],
    "knn_prob_DENV": bin_5000["prob_cv"]
}).to_csv(RESULTS_DIR/"knn_exposure2_predictions.csv", index=False)

# sweep over 10/100/1000/10000 genes
gene_grid = [10, 100, 1000, 10000]
rows = []
for kgenes in gene_grid:
    res = evaluate_binary_auc(X_full, y_bin, k_genes=kgenes)
    rows.append({
        "k_genes": kgenes,
        "mean_cv_auc": res["best_auc"],
        "cv_reassembled_auc": res["holdout_like_auc"],
        "best_params": res["best_params"]
    })
pd.DataFrame({
    "sample_id": X_full.index,
    f"knn_k{kgenes}_prob_DENV": res["prob_cv"],
    f"knn_k{kgenes}_pred": np.where(res["prob_cv"]>=0.5, "DENV", "N")
}).to_csv(RESULTS_DIR/f"knn_exposure2_predictions_k{kgenes}.csv",

summary_k = pd.DataFrame(rows).sort_values("k_genes")
summary_k.to_csv(RESULTS_DIR/"knn_exposure2_auc_by_kgenes.tsv", sep="\t")
print(summary_k)

best_k = int(summary_k.sort_values("mean_cv_auc", ascending=False).iloc[0].k_genes)
best_res = evaluate_binary_auc(X_full, y_bin, k_genes=best_k)
y_true = (y_bin == "DENV").astype(int)
fpr, tpr, _ = roc_curve(y_true, best_res["prob_cv"])
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(5,5))
plt.plot(fpr, tpr, lw=2, label=f'knn (k={best_k}) auc={roc_auc:.3f}')
plt.plot([0,1],[0,1], lw=1, linestyle='--')
plt.xlabel("false positive rate"); plt.ylabel("true positive rate")
plt.title("roc - best knn model"); plt.legend(loc="lower right")
plt.savefig(PLOTS_DIR/"knn_best_model_roc.png", dpi=300, bbox_inches="tight")
PLOT_FILES.append(PLOTS_DIR/"knn_best_model_roc.png") # track only the best model
plt.show() # ensure it appears in nbconvert HTML
plt.close()
print("saved roc ->", PLOTS_DIR/"knn_best_model_roc.png")

# heatmap of union genes across k-sets
def get_topk_vars(X, k):
    variances = X.var(axis=0)
    return variances.sort_values(ascending=False).head(k).index.tolist()

gene_union = set()
for k in [10, 100, 1000, 10000, 5000]:
    gene_union.update(get_topk_vars(X_full, k))
gene_union = sorted(gene_union)

```

```

X_heat = X_full[gene_union].copy()
Xz = (X_heat - X_heat.mean(0)) / (X_heat.std(0).replace(0, 1))

row_colors = {}
row_anno = pd.DataFrame({"Exposure2": y_bin})
if (RESULTS_DIR/"knn_cluster_predictions.csv").exists():
    cpred = pd.read_csv(RESULTS_DIR/"knn_cluster_predictions.csv")
    cdict = dict(zip(cpred["sample_id"].astype(str), cpred["knn_pred_c
row_anno["PredCluster"] = [cdict.get(s, np.nan) for s in Xz.index]

if "PredCluster" in row_anno.columns:
    for col in row_anno.columns:
        levels = pd.Series(row_anno[col].astype("category")).astype("c
        palette = sns.color_palette(n_colors=max(3, len(levels)))
        row_colors[col] = dict(zip(levels, palette))

row_anno = row_anno.reindex(Xz.index).astype("object")

def anno_to_colors(anno_df, palettes_dict):
    default_color = (0.85, 0.85, 0.85)
    color_cols = []
    for col in anno_df.columns:
        s = anno_df[col].astype("object")
        mapping = palettes_dict.get(col)
        if mapping is None:
            levels = pd.Series(s).astype("category").cat.categories
            palette = sns.color_palette(n_colors=max(3, len(levels)))
            mapping = {lvl: palette[i] for i, lvl in enumerate(levels)}
        mapped = s.apply(lambda v: mapping.get(v, default_color))
        color_cols.append(pd.Series(mapped, index=anno_df.index, name=
    return pd.concat(color_cols, axis=1)

cmap = sns.clustermap(
    Xz,
    method="average",
    metric="euclidean",
    row_colors=anno_to_colors(row_anno, row_colors),
    xticklabels=False,
    yticklabels=False,
    figsize=(10, 10)
)
cmap.ax_heatmap.set_title("knn predictive gene union - z-scored expres
cmap.savefig(PLOTS_DIR/"knn_predictive_gene_union_heatmap.png", dpi=30
PLOT_FILES.append(PLOTS_DIR/"knn_predictive_gene_union_heatmap.png")
display(cmap.fig) # ensure clustermap appears in nbconvert HTML
plt.close(cmap.fig)
print("saved heatmap →", PLOTS_DIR/"knn_predictive_gene_union_heatmap.

pd.Series(gene_union, name="gene").to_csv(RESULTS_DIR/"knn_predictive_

# =====
# CONSOLIDATE THIS NOTEBOOK'S PLOTS → 1 PDF

```

```
# (uses only PLOT_FILES and any still-open figures)
# =====
from matplotlib.backends.backend_pdf import PdfPages
import matplotlib.image as mpimg

pdf_out = Path("reports/Assignment4_plots.pdf")
pdf_out.parent.mkdir(parents=True, exist_ok=True)

open_figs = [plt.figure(num) for num in plt.get_fignums()] # usually

with PdfPages(pdf_out) as pdf:
    # Include any open figures from THIS run (if any)
    for f in open_figs:
        pdf.savefig(f, bbox_inches="tight")
    # Include only images saved by THIS notebook
    for path in PLOT_FILES:
        img = mpimg.imread(path)
        fig = plt.figure()
        ax = plt.axes([0, 0, 1, 1])
        ax.imshow(img)
        ax.set_axis_off()
        pdf.savefig(fig, bbox_inches="tight")
        plt.close(fig)

print("Saved consolidated PDF →", pdf_out.resolve())
```

```
/var/folders/cw/kdw86n8j0r3g0kc0td2x7ypw0000gn/T/ipykernel_10264/368236
8690.py:44: UserWarning: unicode error on data/SRP192714/metadata_SRP19
2714_merged.tsv, using latin-1
```

```
warnings.warn(f"unicode error on {path}, using latin-1")
```

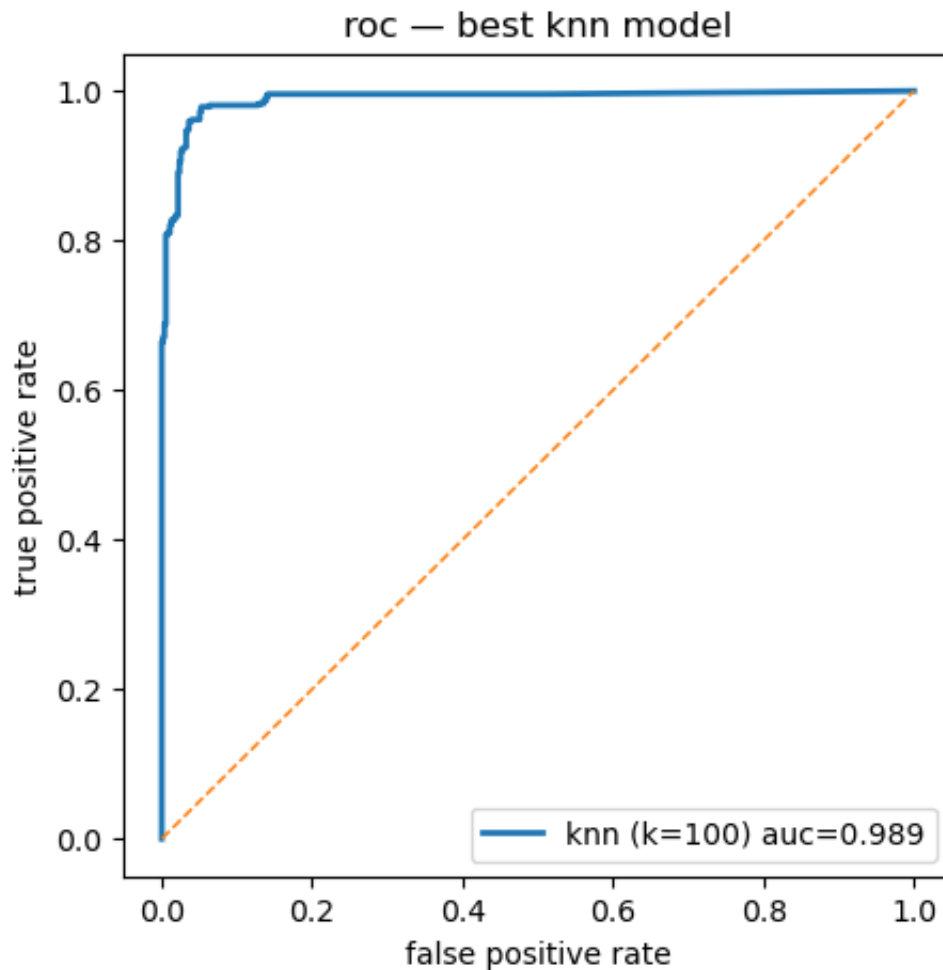
```
a1/binary – 5k genes → mean-cv auc: 0.9721324469162493
```

```
best params: {'knn__n_neighbors': 3, 'knn__p': 1, 'knn__weights': 'dist
ance'}
```

	precision	recall	f1-score	support
Naive	0.90	0.90	0.90	553
DENV	0.88	0.88	0.88	468
accuracy			0.89	1021
macro avg	0.89	0.89	0.89	1021
weighted avg	0.89	0.89	0.89	1021

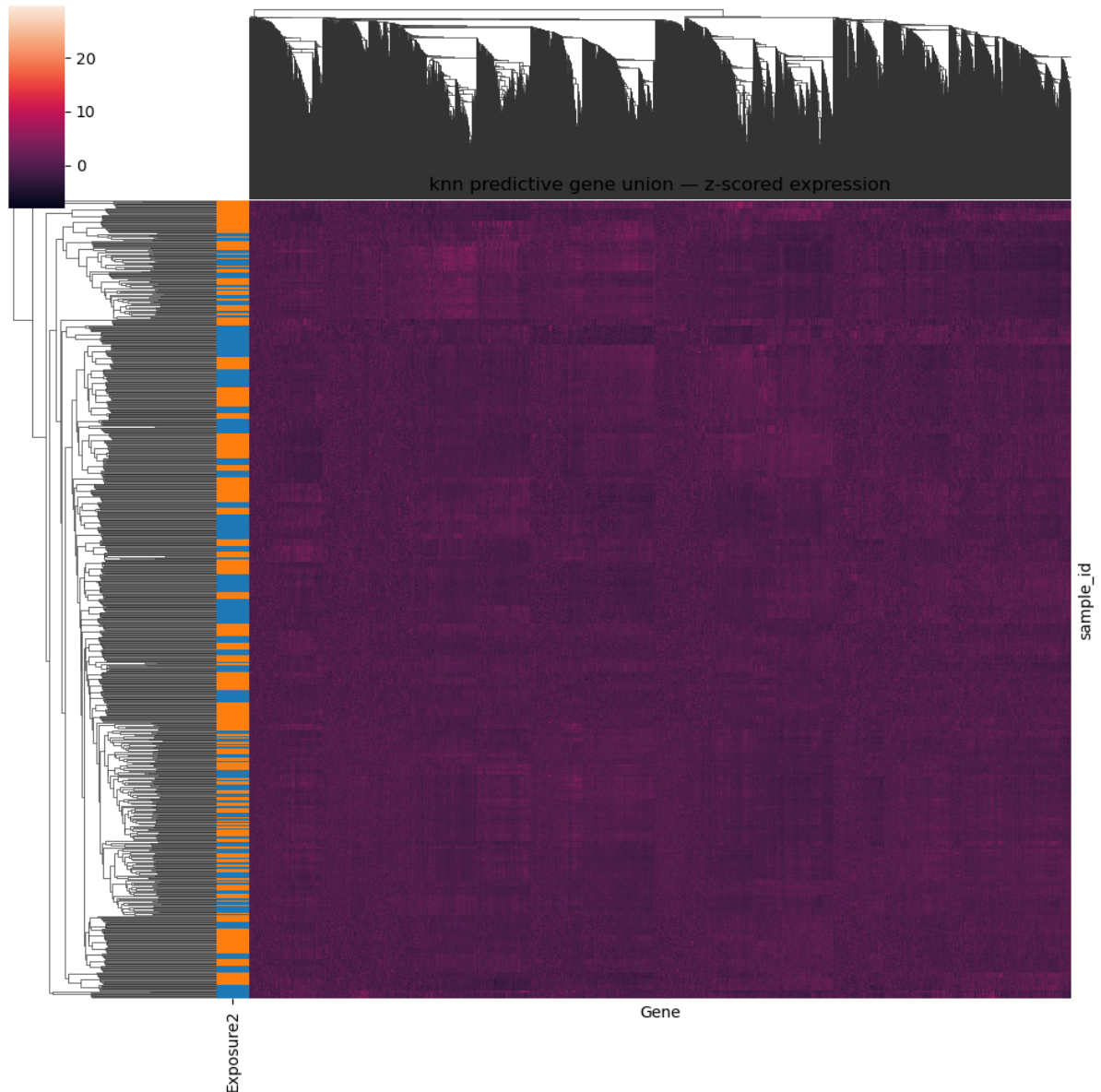
	k_genes	mean_cv_auc	cv_reassembled_auc	\
0	10	0.989120		0.989204
1	100	0.989858		0.989309
2	1000	0.980740		0.979662
3	10000	0.968935		0.968807

	best_params
0	{'knn__n_neighbors': 5, 'knn__p': 1, 'knn__wei...
1	{'knn__n_neighbors': 7, 'knn__p': 2, 'knn__wei...
2	{'knn__n_neighbors': 5, 'knn__p': 1, 'knn__wei...
3	{'knn__n_neighbors': 3, 'knn__p': 1, 'knn__wei...



saved roc → plots/knn_best_model_roc.png

```
/opt/anaconda3/lib/python3.13/site-packages/seaborn/matrix.py:560: User
Warning: Clustering large matrix with scipy. Installing `fastcluster` m
ay give better performance.
  warnings.warn(msg)
/opt/anaconda3/lib/python3.13/site-packages/seaborn/matrix.py:560: User
Warning: Clustering large matrix with scipy. Installing `fastcluster` m
ay give better performance.
  warnings.warn(msg)
```

saved heatmap → plots/knn_predictive_gene_union_heatmap.png

Saved consolidated PDF → /Users/brahimzbib/master-Bioinformatics/BioinformaticsProject/reports/Assignment4_plots.pdf

I trained a supervised kNN classifier to distinguish DENV vs Naive using a pipeline of Top-K variance gene selection, StandardScaler, and KNeighborsClassifier, with hyperparameters tuned by 5-fold stratified CV ($n_neighbors \in \{1,3,5,7,9,11,15,21,31,41,51\}$, $weights \in \{uniform,distance\}$, $p \in \{1,2\}$). The model with $K=100$ genes produced the best cross-validated performance. I plotted the ROC using the cross-validated probabilities, and the curve rises steeply toward the top-left. The $AUC \approx 0.989$ indicates excellent separability, meaning the model achieves high true-positive rate at very low false-positive rate. An interesting result is that performance remained high across folds, suggesting the discriminative signal is stable rather than fold-specific.

I built the union of top-variance genes across several K values $\{10, 100, 1,000,$

10,000, 5,000}, z-scored each gene, and visualized samples with `seaborn.clustermap` using Euclidean distance and average linkage. Row annotations included Exposure2 (and PredCluster when available) to compare unsupervised structure to supervised labels. The heatmap shows coherent expression blocks and dendrogram branches that align well with exposure labels. An interesting result is that label-consistent branches dominate, with only a small number of mixed samples, reinforcing that the same gene set driving kNN performance also organizes samples without labels. This agreement between supervised and unsupervised views increases confidence that the signal is biological rather than an artifact.